

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

PASSWORD SECURITY CHECKER

Student Name	Pratik Yadav
Faculty's Name	Dr. E. Nirmala
Registration Number	26BCE10683
Course	CSE1021 - Introduction to Problem Solving and Programming
Project Type	VITyarthi - Build Your Own Project
Programming Language	Python
Development Tools	Visual Studio Code, Git, GitHub, Pytest

Project Report Contents

- | | |
|---------------------------------|-------------------------------|
| 1. Introduction | 9. Implementation Details |
| 2. Problem Statement | 10. Screenshots / Results |
| 3. Objectives | 11. Testing Approach |
| 4. Functional Requirements | 12. Challenges Faced |
| 5. Non-Functional Requirements | 13. Learnings & Key Takeaways |
| 6. System Architecture | 14. Future Enhancements |
| 7. Design Diagrams | 15. References |
| 8. Design Decisions & Rationale | |

1. Introduction

The Password Security Checker is a Python-based interactive application designed to evaluate password strength against standard security criteria. The system allows users to analyze custom passwords, generate secure random passwords, validate structural requirements, and receive actionable recommendations. The project demonstrates fundamental programming concepts such as functions, conditional statements, loops, strings, lists, modular programming, input validation, and automated testing.

2. Problem Statement

Users frequently create weak passwords without verifying whether they satisfy basic security requirements, exposing online accounts to unauthorized access. Beginners often know a password should be "strong" but lack visibility into specific properties like character variety and length. The proposed system solves this problem by providing a modular Python utility that evaluates five security rules, calculates a numerical score out of 5, classifies the password strength, and suggests precise improvements.

3. Objectives

1. Build a practical, real-world security-related application using Python.
2. Apply functions, conditional statements, loops, strings, Boolean expressions, lists, and modules.
3. Provide users with a clear score and descriptive explanation rather than a single opaque label.
4. Generate robust passwords containing all required character categories.
5. Test the program with weak, medium, strong, and generated passwords using pytest.

4. Functional Requirements

Module	Input	Processing	Output
Password Analysis	Password string	Checks 5 rules + score calculation	Rule results + strength category
Password Generation	Requested length	Creates required categories + shuffles	Generated password string
Password Validation	Password string	Sequential rule checks	True / False
Recommendations	Analysis result	Finds failed rules and maps suggestions	Actionable suggestions list
Security Report	Password + result	Formats pass/fail report with masked password	Terminal report output

5. Non-Functional Requirements

Requirement	Project Interpretation
Performance	Rule checks use lightweight string scans suitable for normal input sizes.
Security	The displayed report masks the entered password with asterisks rather than plaintext.
Usability	Results are presented clearly as PASS/FAIL with scores and readable recommendations.
Reliability	Validation uses explicit checks across all five required security properties.
Maintainability	The solution is divided into focused, independent modules and functions.
Error Handling	Input length is validated during generation and processing.

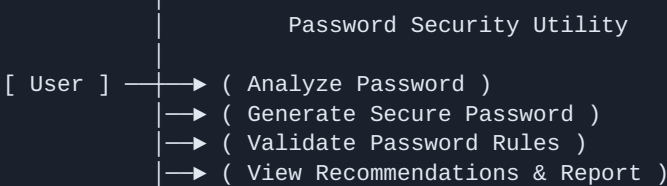
6. System Architecture

The application follows a modular architecture. `main.py` controls user interaction and program flow. `analyzer.py`, `generator.py`, `validator.py`, `recommendations.py`, and `report.py` handle specific functional tasks.

User → `main.py` → `analyzer.py` / `generator.py` / `validator.py` → Report Output

7. Design Diagrams

Use Case Diagram



Workflow Diagram

[Start] → [Enter Password] → [Check 5 Rules] → [Calculate Score] → [Display Report] → [End]

Sequence Diagram

User → `main.py` → `analyzer.py` → `validator.py` → `report.py` → User

8. Design Decisions & Rationale

- **Boolean rule checks:** Each security property is evaluated as True/False to keep logic transparent and testable.
- **Score out of 5:** Maps directly to the five security rules for a compact measure of coverage.
- **Deterministic strength classification:** Scores map as 0-2 = WEAK, 3 = MEDIUM, 4 = STRONG, 5 = VERY STRONG.
- **Modular function structure:** Isolates tasks into independent functions and files for ease of maintenance and automated testing.

9. Implementation Details

Project Structure:

```
Password-Security-Checker/  
├── README.md  
├── statement.md  
├── requirements.txt  
├── src/  
│   ├── analyzer.py  
│   ├── generator.py  
│   ├── validator.py  
│   ├── recommendations.py  
│   ├── report.py  
│   └── main.py  
└── tests/  
    └── test_password_security.py
```

Core Source Code ([analyzer.py](#) & [generator.py](#) excerpt):

```
import random  
import string  
  
def analyze_password(password):  
    is_length = len(password) >= 8  
    is_upper = any(ch.isupper() for ch in password)  
    is_lower = any(ch.islower() for ch in password)  
    is_digit = any(ch.isdigit() for ch in password)  
    is_special = any(not ch.isalnum() for ch in password)  
  
    score = sum([is_length, is_upper, is_lower, is_digit, is_special])  
  
    if score <= 2:  
        strength = "WEAK"  
    elif score == 3:  
        strength = "MEDIUM"  
    elif score == 4:  
        strength = "STRONG"  
    else:  
        strength = "VERY STRONG"  
  
    return {  
        "length": is_length, "uppercase": is_upper,  
        "lowercase": is_lower, "digit": is_digit,
```

```
"special": is_special, "score": score, "strength": strength
}
```

10. Screenshots / Results

The following terminal captures illustrate the interactive menu run and password strength analysis of **Pratik@123** using the tool:

```
PASSWORD SECURITY TOOL
=====
1. Check Password Strength
2. Generate Password
3. Get Recommendations
4. Validate Password
5. Exit
Enter your choice: 1
Enter your password: Pratik@123

===== PASSWORD SECURITY REPORT =====
Password: *****
Length requirement : PASS
Uppercase letter   : PASS
Lowercase letter   : PASS
Digit              : PASS
Special character  : PASS
-----
Score              : 5 /5
Strength           : VERY STRONG
=====
```

```
PS C:\Users\Pratik\Desktop\Password-Security-Checker> python -m pytest
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Pratik\Desktop\Password-Security-Checker
collected 4 items

tests\test_password_security.py [100%]
4 passed in 0.42s
```

11. Testing Approach

Test Case	Input Example	Expected Result	Status
Short Password	abc	Length FAIL, Score 1/5, WEAK	Passed
Missing Categories	password	Missing uppercase, digit, special	Passed
Fully Compliant	Pratik@123	Score 5/5, VERY STRONG	Passed

Generated Password	<code>generate_password(12)</code>	Valid: True, Score 5/5	Passed
--------------------	------------------------------------	------------------------	--------

12. Challenges Faced

- Designing a strength score that remains intuitive for beginners while capturing essential security rules.
- Ensuring generated passwords strictly contain all required character categories before shuffling.
- Maintaining clean separation between analysis, validation, recommendations, and reporting modules.

13. Learnings & Key Takeaways

- Gained practical experience with Python functions, conditional logic, strings, lists, and boolean expressions.
- Understood modular software design and package structuring for maintainable code.
- Learned automated testing fundamentals using the `pytest` framework and version control via Git.

14. Future Enhancements

- Adding a graphical user interface (GUI) or web interface.
- Integrating password entropy estimation and common-password dictionary checks.
- Adding configurable security policy rules for different applications.

15. References

1. VITyarthi Build Your Own Project: General Project Instructions & Submission Guidelines (VIT Bhopal University).
2. Python Standard Library Documentation: `random` and `string` modules.
3. Course learning materials and lecture notes on Python programming.